Lab 1.3 – AI Assisted Coding (Fibonacci Tasks)

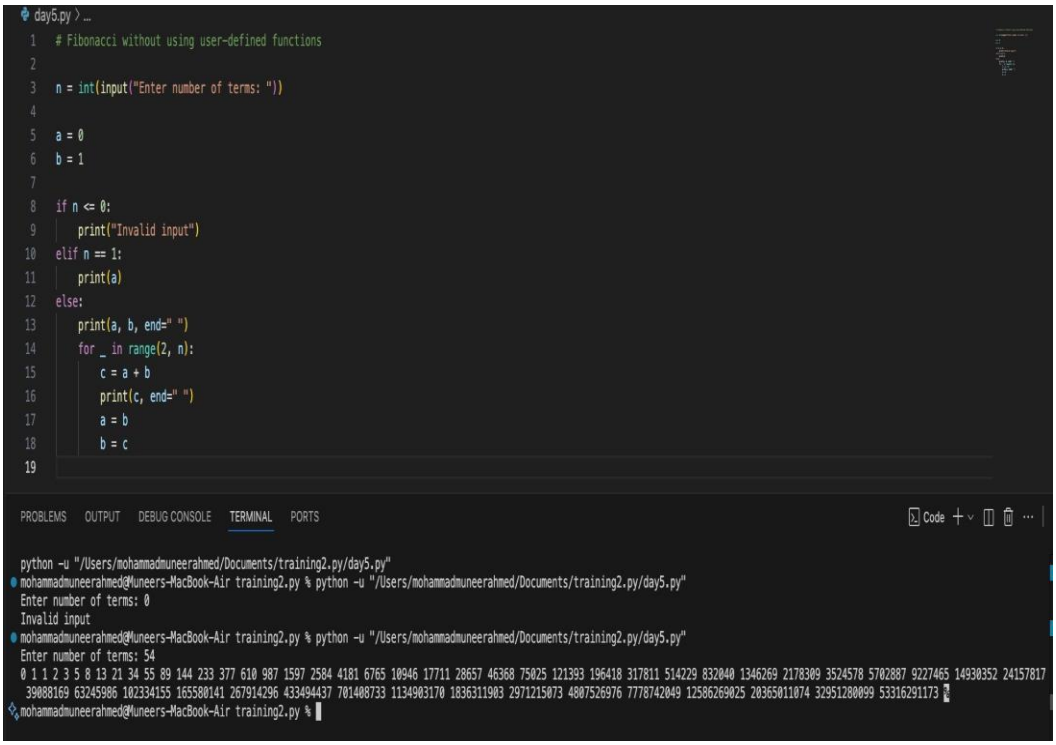
Name: Daniel Anthonello

Roll No: 2303A51438

Task 1 – Fibonacci Without Functions (Procedural)

Algorithm:
1. Read integer n
2. Initialize a=0, b=1
3. Handle base cases
4. Loop and compute next = a+b
5. Update variables

Pseudocode:
INPUT n
a←0,b←1
FOR i from 0 to n
PRINT a
a,b←b,a+b



Task 2 – Optimized Fibonacci

Algorithm:
1. Read n
2. Use tuple swap update
3. Print each term

Pseudocode:
INPUT n
a,b←0,1
REPEAT n times
PRINT a
a,b←b,a+b

day5.py > ...

```python
# Optimized Fibonacci without redundant variables

n = int(input("Enter number of terms: "))

a, b = 0, 1

for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
mohammadmuneerahmed@Muneers-MacBook-Air training2.py % python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
Enter number of terms: 4
0 1 1 2 %
mohammadmuneerahmed@Muneers-MacBook-Air training2.py %
```

Task 3 – Modular Fibonacci Using Function

Algorithm:
1. Define function
2. Loop and append
3. Return sequence

Pseudocode:
FUNCTION fib(n)
a,b←0,1
list←[]
LOOP n
append a
a,b←b,a+b
RETURN list

day5.py > ...

```python
# Fibonacci using modular design

def fibonacci(n):
    seq = []
    a, b = 0, 1
    for _ in range(n):
        seq.append(a)
        a, b = b, a + b
    return seq


n = int(input("Enter number of terms: "))
print(*fibonacci(n))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
mohammadmuneerahmed@Muneers-MacBook-Air training2.py % python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
Enter number of terms: 6
0 1 1 2 3 5
mohammadmuneerahmed@Muneers-MacBook-Air training2.py %
```

Task 4 – Procedural vs Modular Comparison

Algorithm:
1. Evaluate reuse
2. Evaluate clarity
3. Evaluate debugging

Pseudocode:
COMPARE procedural vs modular
REPORT differences

Procedural Fibonacci code is acceptable for quick scripts and demonstrations but fails in reuse, testing, and maintainability. Modular Fibonacci code separates concerns, supports reuse, simplifies debugging, and scales to larger applications.

Task 5 – Iterative vs Recursive Fibonacci

Algorithm:
Iterative loop method
Recursive base+call method
Pseudocode:
ITERATIVE loop update
RECURSIVE fib(n-1)+fib(n-2)

day5.py > ...

```python
def fib_recursive(n):
    if n <= 1:
        return n
    return fib_recursive(n-1) + fib_recursive(n-2)


terms = int(input("Enter number of terms: "))
for i in range(terms):
    print(fib_recursive(i), end=" ")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
mohammadmuneerahmed@Muneers-MacBook-Air training2.py % python -u "/Users/mohammadmuneerahmed/Documents/training2.py/day5.py"
Enter number of terms: 5
0 1 1 2 3 %
mohammadmuneerahmed@Muneers-MacBook-Air training2.py %
```